

1 Fault-Detector Metrics: Confusion Matrix

Fault-detector $fd(t)$: function taking system observations $o(t)$ as input, outputting a conjecture of whether a fault is present (binary: $FD=1$ fault, $FD=0$ no fault).

Input $o(t)$ can be a **single observation** or a **sequence** of past observations — you define.

System has 2 true states: $S=0$ (normal) and $S=1$ (faulty) $\Rightarrow$ 4 outcome combinations (confusion matrix below).

	Negatives ($FD=0$)	Positives ($FD=1$)
$S=0$	TNR $\Pr(FD=0 \mid S=0)$	FPR $\Pr(FD=1 \mid S=0)$
$S=1$	FNR $\Pr(FD=0 \mid S=1)$	TPR $\Pr(FD=1 \mid S=1)$

$$TNR + FPR = 1$$

$$FNR + TPR = 1$$

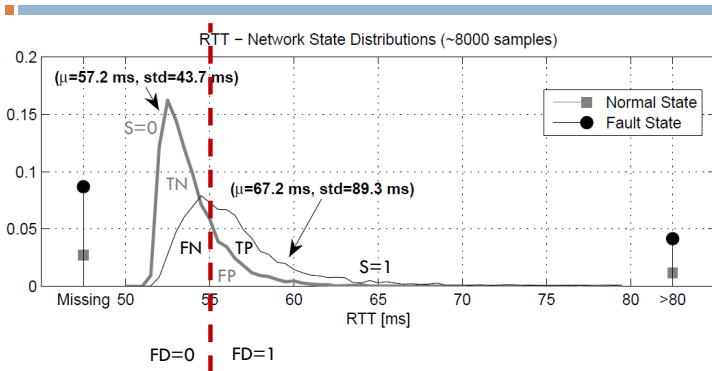
Accuracy:

$$Acc = TPR \cdot \Pr(S=1) + TNR \cdot \Pr(S=0)$$

Depends on how often system is faulty.

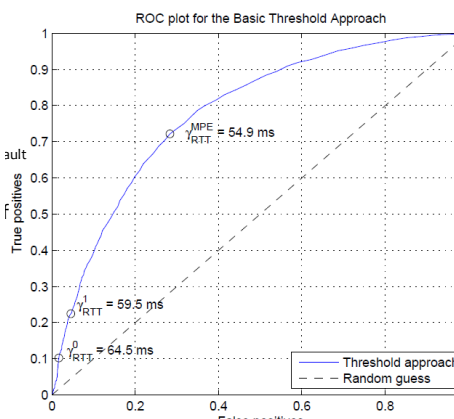
2 Example: Congestion Detection & ROC Curve

Congestion detector: $FD(t_i) = \mathbf{1}(RTT_i > \gamma)$ — declare congestion ($FD=1$) when RTT exceeds threshold γ , normal ($FD=0$) otherwise.



Example: Thresholds = 55ms

To set γ precisely we can use the **ROC curve** — sweeping γ traces a curve in (FPR, TPR) space, letting us choose the best operating point for the required trade-off.



Congestion detector: $FD(t_i) = \mathbf{1}(RTT_i > \gamma)$

RTT distributions for $S=0$ (normal) and $S=1$ (fault) **overlap** $\Rightarrow$ no perfect threshold exists.

At $\gamma=55$ ms (dashed line):

- Left of γ : $FD=0 \Rightarrow$ **TN** (if $S=0$) or **FN** (if $S=1$)
- Right of γ : $FD=1 \Rightarrow$ **FP** (if $S=0$) or **TP** (if $S=1$)

Moving γ right: $FPR \downarrow$ and $TPR \downarrow$ simultaneously.

ROC (Receiver Operating Characteristic):

sweep $\gamma \Rightarrow$ plot (FPR, TPR) pairs.

- (0,1) **top-left**: ideal — $TPR=1$, $FPR=0$
- Diagonal**: random guess (useless)
- $\gamma^{MPE}=54.9$ ms: min. prob. error point
- Choose γ on curve to match required FPR/TPR trade-off

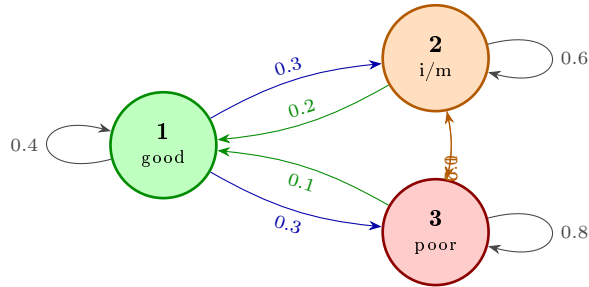
Why we use HMM (Need to model uncertainty of fault knowledge)

Why HMM? The true system state (faulty/normal) is **not directly observable** — we only ever get indirect measurements such as RTT. We must therefore **infer** the hidden fault state purely from these observations.

HMM formalises exactly this: a hidden Markov state evolves over time, emitting observable symbols — allowing us to estimate the most likely underlying fault state from what we can actually measure.

3 HMM Definition: $\langle E, V, \pi_1, \underline{P}, \underline{B} \rangle$

Example: we can only observe whether a packet was received correctly or with error (c/e) — we want to determine the hidden wireless link state (good, intermediate, poor).



Emission matrix $\underline{B}$

$$b_{ij} = \Pr(O_k=j \mid X_k=i)$$

State	C (correct)	E (error)
good	≈ 1	10^{-5}
i/m	0.98	0.02
poor	0.80	0.20

Each row sums to 1. Hidden state drives *which* observation symbol is generated.

$\text{HMM} = \langle E, V, \pi_1, \underline{P}, \underline{B} \rangle$

- E : **hidden state space** (e.g. good, i/m, poor) — N states
- V : **observation symbol set** (e.g. $\{c, e\}$) — M symbols
- π_1 : **initial state probabilities** at step 1
- $\underline{P}$: $N \times N$ **transition matrix**, $p_{ij} = \Pr(X_{k+1}=j \mid X_k=i)$
- $\underline{B}$: $N \times M$ **emission matrix**, $b_{ij} = \Pr(O_k=j \mid X_k=i)$

Note Discrete Markov is a special case: each column of $\underline{B}$ has at most one non-zero entry (state reveals itself exactly).

4 Three HMM Algorithms

1. Forward

Find: $\Pr(\underline{Q}=\underline{q} \mid \text{model})$

How: smart reuse of previous step's results (DP) — sums over all hidden paths without recomputing, giving $O(N^2T)$ instead of brute-force $O(N^T)$.

2. Viterbi

Use when: you want the *most likely hidden state at each step* — e.g. fault diagnosis from RTT/RSSI observations.

Does: same DP as Forward but tracks the *max-prob* path; backtracks to recover the state sequence.

3. Baum-Welch (*not examined*)

Assume: state space E and symbol set V are known.

Find: $\underline{P}$ and $\underline{B}$ from obs. data alone — EM-style iteration, converges to a *local* optimum.

5 Application: Fault Detection

Single fault (congestion): hidden states = {normal, congested}; obs = RTT (discretised). Fit $\underline{B}$ from measured RTT distributions per state, $\underline{P}$ from fault-occurrence rates. Run **Viterbi** on RTT seq. → fault state at each step. Outperforms threshold: uses *sequence context* so single outliers don't trigger.

Multi-fault (wireless + congestion): 4 states {00,10,01,11} encoding link × congestion condition. Two observations per step: RTT (both faults) + RSSI (wireless only). $\underline{B}$ becomes $4 \times M$ joint emission matrix. Viterbi output = joint fault state at every time step → separate alarms per fault type.

reactivity

Reactivity (good to know): distribution of time between fault occurrence and first true positive (given detected).

